

EMBEDDED SYSTEMS

Complete 7-Day Study Guide

■ DAY 1 ■

Introduction to Embedded Systems

D1

D2

D3

D4

D5

D6

D7

1. What is an Embedded System?

An **Embedded System** is a dedicated computer system designed to perform one or a few specific functions, often within a larger system. Unlike general-purpose computers, embedded systems are optimized for the task they perform — trading flexibility for efficiency, size, and reliability.

Key Characteristics:

Dedicated Function	Performs a specific, pre-defined task (e.g., controlling a thermostat, managing ABS brakes).
Real-Time Operation	Must respond to inputs within strict time constraints (hard/soft real-time).
Resource Constrained	Limited CPU speed, RAM, ROM, and power budget compared to general-purpose computers.
Reliability	Must operate continuously without failure; often deployed in critical environments.
Low Power	Battery-operated devices require power-efficient design.
Small Form Factor	Compact hardware footprint — PCB size matters in consumer electronics.

Real-World Examples:

Domain	Embedded System Example	Key Constraint
Automotive	ABS, Engine Control Unit (ECU)	Hard Real-Time
Consumer Elec.	Smart TV, Washing Machine	Low Cost
Medical	Pacemaker, Insulin Pump	Ultra-Reliable
Industrial	PLC, Robotic Arm Controller	Deterministic
IoT / Wearable	Smart Watch, Fitness Tracker	Ultra Low Power
Aerospace	Flight Control Computer	Fault Tolerant

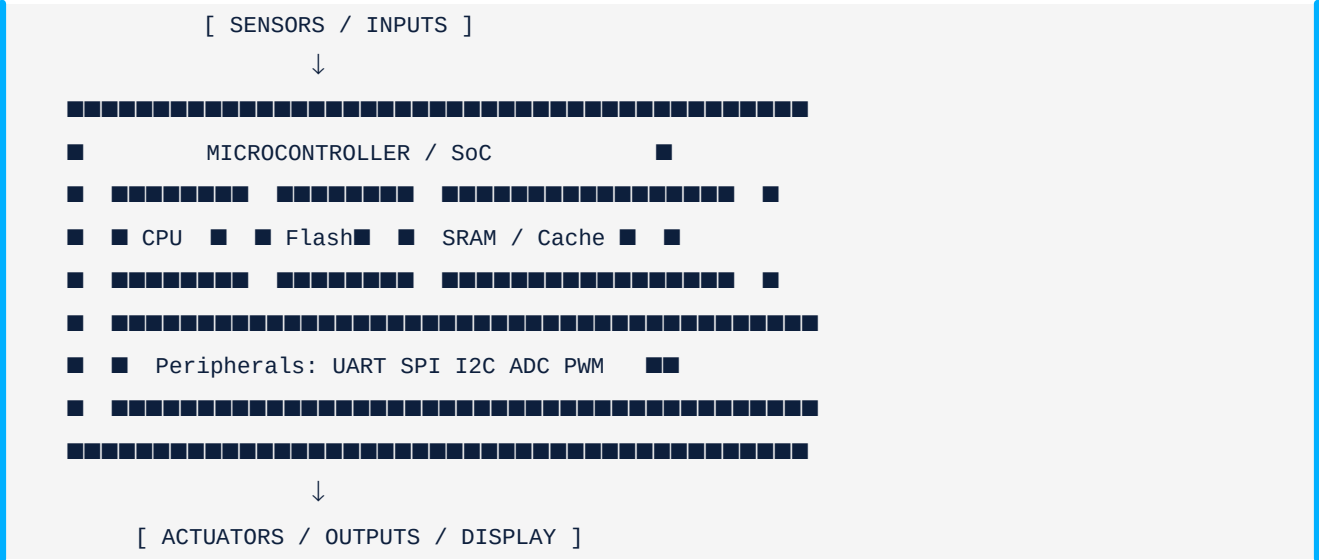
2. Basic Architecture of an Embedded System

Every embedded system — from a simple 8-bit microcontroller to a complex SoC — shares a common architectural template. Understanding this template is the foundation for all hardware/software co-design work.

Core Hardware Blocks:

Processor (CPU/MCU/DSP)	The brain. Executes instructions. Examples: ARM Cortex-M, AVR ATmega, PIC, RISC-V.
Memory	Program memory (Flash/ROM) stores code; Data memory (SRAM) stores variables. Harvard vs Von Neumann.
Input Peripherals	Sensors (temperature, pressure), buttons, ADC, UART RX, SPI MISO, I2C SDA.
Output Peripherals	Actuators, displays, LEDs, DAC, PWM motor drivers, UART TX.
Communication Interfaces	UART, SPI, I2C, CAN, USB, Ethernet, Wi-Fi, Bluetooth for data exchange.
Clock & Timers	System clock (crystal/oscillator), watchdog timer, hardware timers for PWM/scheduling.
Power Supply Unit	Regulators (LDO/SMPS) to convert battery/mains voltage to 3.3V/5V. Sleep modes for power saving.
Bus Architecture	Address bus, Data bus, Control bus — links CPU to memory and peripherals.

Simplified Block Diagram:



3. Microcontroller vs Microprocessor

Feature	Microcontroller (MCU)	Microprocessor (MPU)
Integration	CPU + RAM + Flash + I/O on chip	CPU only; needs external RAM/ROM

Cost	Very low (■20 – ■500)	Higher (■200 – ■5000+)
Power	Ultra-low (µA sleep modes)	Higher power consumption
Speed	8 MHz – 600 MHz typical	GHz range (ARM Cortex-A, x86)
OS Support	Bare-metal / RTOS	Linux, Windows, Android
Use Case	Sensor nodes, appliances, IoT	Smartphones, laptops, gateways
Programming	C, Assembly, MicroPython	C/C++, Python, Java, etc.
Examples	STM32, ATmega328, ESP32	Raspberry Pi CM4, i.MX6, Snapdragon

■ *Rule of Thumb: Use a Microcontroller for standalone sensor/actuator tasks. Use a Microprocessor (SoC running Linux) when you need networking stacks, filesystems, or complex UI.*

4. Memory Organisation

Understanding where data and code live is critical for writing efficient embedded firmware. The two dominant architectures differ in how they connect CPU to memory.

Harvard vs Von Neumann Architecture:

Aspect	Harvard Architecture	Von Neumann Architecture
Bus Structure	Separate buses for data & code	Single shared bus
Speed	Faster (parallel fetch+execute)	Bottleneck at shared bus
Flexibility	Less flexible	More flexible
Used In	AVR, PIC, DSP chips	x86, ARM Cortex-M (modified)
Security	Code cannot overwrite itself	Code injection risk

Memory Types in a Typical MCU:

Flash (Program Memory)	Non-volatile. Stores compiled firmware. Typically 16 KB – 2 MB. Survives power-off. Erase before write (page-wise).
SRAM (Data Memory)	Volatile. Stores stack, heap, global variables at runtime. Fast. Typically 2 KB – 512 KB in MCUs.
EEPROM	Non-volatile byte-addressable storage for configuration data. Slow write (~3 ms/byte). Limited erase cycles (~100K).
ROM / OTP	Factory-programmed bootloader or calibration data. Cannot be changed in field.
External RAM (SDRAM)	Used in MPU-based systems for large buffers, OS heap. Connected via memory bus or QSPI.
Cache (L1/L2)	Fast on-chip buffer (Harvard MCUs may not have it; Cortex-A series does). Reduces latency for repeated access.

Typical ARM Cortex-M Memory Map:

Address Range	Region	Description
0x0000 0000 - 0x1FFF FFFF	Code	Flash / ROM — firmware lives here
0x2000 0000 - 0x3FFF FFFF	SRAM	On-chip RAM — stack, heap, data
0x4000 0000 - 0x5FFF FFFF	Peripheral	Memory-mapped I/O registers
0x6000 0000 - 0x9FFF FFFF	External RAM	SDRAM / PSRAM (if present)
0xA000 0000 - 0xDFFF FFFF	External Device	External Flash / NOR Flash
0xE000 0000 - 0xFFFF FFFF	System	CoreSight debug, NVIC, SysTick

5. Input / Output — GPIO Fundamentals

GPIO (General Purpose Input/Output) pins are the most basic peripheral. Each pin can be configured as input or output and driven/read via memory-mapped registers.

Key GPIO Registers (STM32 naming):

DDRx / MODER	Direction register — sets pin as input (0) or output (1).
PORTx / ODR	Output Data Register — write 1/0 to drive the pin HIGH or LOW.
PINx / IDR	Input Data Register — read the current logic level on the pin.
PUPDRx	Pull-up/Pull-down Register — enables internal resistors to avoid floating inputs.
OSPEEDR	Output Speed Register — controls slew rate (Low/Medium/High/Very High).
OTYPER	Output Type Register — Push-Pull (default) or Open-Drain mode.

Sample Code — LED Blink (STM32 Bare-Metal):

```
// Enable GPIOA clock
RCC->AHB1ENR |= RCC_AHB1ENR_GPIOAEN;

// Set PA5 as output (MODER bits 11:10 = 01)
GPIOA->MODER &= ~(0x3 << (5 * 2)); // clear
GPIOA->MODER |= (0x1 << (5 * 2)); // set output

while (1) {
    GPIOA->ODR ^= (1 << 5); // toggle LED
    for (volatile int i = 0; i < 500000; i++); // delay
}
```

■■ Always enable the peripheral clock (RCC register) before accessing any peripheral register. Forgetting this is the #1 cause of 'nothing works' in bare-metal programming.

6. Communication Protocols — Overview

Protocol	Type	Speed	Wires	Topology	Use Case
UART	Async	Up to 5 Mbps	2 (TX,RX)	Point-to-Point	GPS, Bluetooth module
SPI	Sync	Up to 50 Mbps	4	Master-Slave	Flash memory, displays
I2C	Sync	100K–3.4 Mbps	2	Multi-master bus	Sensors (temp, IMU)
CAN	Async	1 Mbps (Classic)	2 (diff)	Multi-node bus	Automotive ECU network
USB	Sync	480 Mbps (HS)	4	Host-Device	PC peripherals
Ethernet	Sync	1 Gbps+	4 pairs	Switched LAN	Industrial, IoT gateway

7. Development Tools & Workflow

IDE / Editor	STM32CubeIDE, Keil μ Vision, IAR EWARM, VS Code + PlatformIO, Arduino IDE
Cross-Compiler	arm-none-eabi-gcc (GCC ARM), LLVM/Clang — compiles on host for target MCU
Build System	Make, CMake — manages compilation, linking, and flash programming
Flash Programmer	ST-Link, J-Link, OpenOCD — downloads compiled binary to MCU Flash
Debugger (HW)	J-Link EDU, ST-Link V3 — JTAG/SWD interface for breakpoints and register view
Debugger (SW)	GDB (arm-none-eabi-gdb) + Semihosting / printf over UART
Logic Analyser	Saleae, sigrok — capture and decode UART/SPI/I2C waveforms
Oscilloscope	Verify timing, signal integrity, power supply noise
Emulator / Simulator	QEMU, Proteus, SimulIDE — test firmware without hardware
Version Control	Git — track firmware changes; essential for team projects

Typical Embedded Development Workflow:

Step	Action
1. Requirement Analysis	Define hardware constraints, timing, power budget, interfaces needed.
2. Hardware Selection	Choose MCU/MPU family, peripherals, evaluation board.
3. Schematic & PCB	Design circuit (KiCad/Altium); route PCB; order prototype.
4. Firmware Architecture	Decide bare-metal vs RTOS; define task/ISR structure.
5. Driver Development	Write peripheral drivers (GPIO, UART, SPI...) and test on eval board.
6. Application Logic	Implement main application loop, state machines, algorithms.
7. Integration & Test	Combine hardware + firmware; use logic analyser and debugger.

8. Optimisation	Profile flash/RAM usage; optimise for speed/power; fix timing issues.
9. Validation & QA	Environmental testing (temp, vibration); EMC pre-compliance.
10. Production Release	Golden binary, OTA update mechanism, programming jigs.

8. Key Terminology Glossary

ISR	Interrupt Service Routine — function executed when hardware interrupt fires.
RTOS	Real-Time Operating System — manages tasks with timing guarantees (FreeRTOS, Zephyr).
HAL	Hardware Abstraction Layer — portable API over bare-metal registers.
BSP	Board Support Package — MCU startup code, linker scripts, clock init.
DMA	Direct Memory Access — transfers data without CPU involvement; frees CPU cycles.
WDT	Watchdog Timer — resets MCU if firmware hangs and fails to kick the timer.
JTAG / SWD	Debug interfaces. JTAG: 5 pins. SWD (ARM): 2 pins. Used for flashing & debugging.
PWM	Pulse Width Modulation — digital signal mimicking analog output (motor speed, LED dimming).
ADC	Analog-to-Digital Converter — converts real-world voltage to digital integer.
DAC	Digital-to-Analog Converter — converts digital value to analog voltage.
Bootloader	Small program in fixed Flash address; initialises hardware and loads main firmware.
Linker Script	Controls how code/data sections are placed in Flash and RAM address space.
Volatile	C keyword telling compiler not to cache a variable; essential for hardware registers.
Bare-Metal	Firmware running directly on hardware without an OS; maximum control, minimum overhead.

9. Day-1 Quick Quiz (Self-Test)

Q1	What are three key differences between a microcontroller and a microprocessor?
Q2	Why must the RCC clock be enabled before using a peripheral on STM32?

Q3	Explain the difference between Harvard and Von Neumann memory architecture.
Q4	What register do you write to in order to set a GPIO pin HIGH on an STM32?
Q5	Name two non-volatile memory types used in embedded systems and their key difference.
Q6	Which serial protocol uses only 2 wires, supports multiple devices, and is synchronous?
Q7	What is the purpose of a Watchdog Timer?
Q8	What does the 'volatile' keyword do and why is it critical for embedded firmware?

7-Day Course Roadmap:

Day	Topic
Day 1 ✓	Introduction — Architecture, GPIO, Memory, Protocols
Day 2 →	Interrupts, Timers & PWM
Day 3	UART, SPI, I2C — Deep Dive
Day 4	ADC/DAC & Sensor Interfacing
Day 5	RTOS Fundamentals (FreeRTOS)
Day 6	Power Management & Low-Power Design
Day 7	Bootloaders, OTA & System Design Project